รายงานปฏิบัติการที่ 3

MIME File Transfer over Router-on-a-Stick with Wireshark Analysis

---

วิชา: Computer Network

หัวข้อ: การกำหนดค่า Inter-VLAN Routing, การส่งไฟล์ด้วย MIME Protocol และการวิเคราะห์ Packet ด้วย Wireshark

จัดทำโดย: นายนครินทร์ ณ ระนอง 673380274-7 Section 02

1. บทนำและวัตถุประสงค์

1.1 ภาพรวมของการทดลอง

การทดลองนี้มีวัตถุประสงค์เพื่อผสานความรู้ด้านเครือข่ายคลาสสิกเข้ากับการออกแบบโปรโตคอลในระดับ Application Layer โดยผู้เรียนจะได้กำหนดค่า Inter-VLAN Routing ด้วยเทคนิค Router-on-a-Stick, ทดสอบการส่งไฟล์ด้วยโปรโตคอล MIME ที่สร้างขึ้นเอง (Custom MIME-based Protocol) จากนั้นบันทึกและวิเคราะห์ Packet ด้วย Wireshark

1.2 วัตถุประสงค์การเรียนรู้

- กำหนดค่า Router-on-a-Stick Inter-VLAN Routing บน Router Cisco
- กำหนดค่า VLAN และ Trunking บน Switch Layer 2
- ติดตั้งและรัน MIME Socket Server และ Client ด้วย Python
- บันทึก Traffic ด้วย Wireshark
- ระบุ ARP, VLAN Tags, TCP Three-Way Handshake และข้อมูล Application
- ถอดรหัส JSON Header ใน Packet Payload
- แก้ไขปัญหา (Troubleshoot) ด้านเครือข่ายและ Application
- จัดทำเอกสารอธิบายพฤติกรรมของโปรโตคอลตามชั้น OSI

2. โทโพโลยีและแผนการกำหนดที่อยู่ IP

2.1 ไดอะแกรมโทโพโลยี

การทดลองใช้โทโพโลยีดังนี้:

```
PC1 (VLAN 10) ----\
                   \ access
                    \
                    [S1]---- trunk ----[R1]
                    /
                   / access
PC2 Server (VLAN 20)
```

Router R1 เชื่อมต่อกับ Switch S1 ผ่านลิงก์ Trunk โดย Router ทำหน้าที่เป็น Default Gateway สำหรับทั้งสอง VLAN ผ่าน Subinterface ของพอร์ต G0/0

2.2 ตารางการกำหนดที่อยู่ IP

อุปกรณ์	Interface	IP Address	Default Gateway
R1	G0/0.10	192.168.10.1/24	—
R1	G0/0.20	192.168.20.1/24	—
PC1 (Client)	NIC	192.168.10.10/24	192.168.10.1

PC2 (Server)	NIC	192.168.20.20/24	192.168.20.1

3. ทฤษฎีพื้นฐาน

3.1 Router-on-a-Stick

Router-on-a-Stick คือเทคนิคที่ใช้ Physical Interface เพียงหนึ่งพอร์ตในการรับส่งข้อมูลจากหลาย VLAN ผ่านการแท็ก 802.1Q โดย Router จะสร้าง Subinterface สำหรับแต่ละ VLAN เพื่อทำหน้าที่ Routing ระหว่าง Broadcast Domain

ข้อดี: ลดค่าใช้จ่ายด้านฮาร์ดแวร์ ไม่จำเป็นต้องใช้ Layer 3 Switch ข้อเสีย: Bandwidth ของลิงก์ Trunk เป็น Bottleneck ไม่เหมาะกับ Traffic ปริมาณมาก

3.2 MIME Metadata

MIME (Multipurpose Internet Mail Extensions) ถูกใช้เพื่อระบุประเภทของเนื้อหา (Content Type) เช่น image/png, application/pdf ในการทดลองนี้ โปรโตคอลที่พัฒนาขึ้นเองจะห่อหุ้มไฟล์ด้วย JSON Header ที่อธิบายข้อมูลเมตาได้แก่ mime_type และ size ตามด้วย Binary Payload ของไฟล์จริง

ตัวอย่าง JSON Header:

```
{"mime_type": "image/png", "size": 12345}
```

3.3 ความสำคัญของการออกแบบนี้

รูปแบบ Structured Metadata + Binary Payload over Reliable Transport ปรากฏอยู่ในระบบจริงหลายประเภท เช่น REST API, Storage Services และ Message Brokers การทดลองนี้จึงสะท้อนแนวทางการออกแบบโปรโตคอลที่ใช้ในอุตสาหกรรมจริง

4. ส่วนที่ 1 — การกำหนดค่าอุปกรณ์เครือข่าย

4.1 การกำหนดค่า Router R1

ขั้นตอนการกำหนดค่า Router-on-a-Stick บน R1 มีดังนี้:

```
enable
configure terminal
hostname R1
interface g0/0
  no shutdown
interface g0/0.10
  encapsulation dot1Q 10
  ip address 192.168.10.1 255.255.255.0
interface g0/0.20
  encapsulation dot1Q 20
  ip address 192.168.20.1 255.255.255.0
end
```

```
write memory
```

ผลลัพธ์ที่ได้: *Subinterface G0/0.10* รับผิดชอบ *VLAN 10* และ *G0/0.20* รับผิดชอบ *VLAN 20* *Router* จะทำ *Routing* ระหว่างสอง *Subnet* โดยอัตโนมัติ

(หมายเหตุ: ในสภาพแวดล้อมจริงหรือ Simulator ให้ Screenshot ผลลัพธ์ของคำสั่ง show ip interface brief ในส่วนนี้)

4.2 การกำหนดค่า Switch S1

```
enable
configure terminal
hostname S1
vlan 10
  name CLIENTS
vlan 20
  name SERVERS
interface g0/1
  switchport mode trunk
interface fa0/1
  switchport mode access
  switchport access vlan 10
interface fa0/2
  switchport mode access
  switchport access vlan 20
end
write memory
```

ตรวจสอบการกำหนดค่า Switch ด้วยคำสั่ง:

- show interfaces trunk — ตรวจสอบว่า G0/1 อยู่ในโหมด Trunk และส่ง VLAN 10, 20
- show mac address-table — ดู MAC Address ที่เรียนรู้บนแต่ละพอร์ต
- show vlan brief — ยืนยันการสร้าง VLAN ทั้งสองและ Port Assignment

4.3 การกำหนดค่า Host

อุปกรณ์	IP Address	Subnet Mask	Default Gateway
PC1 (Client)	192.168.10.10	255.255.255.0	192.168.10.1
PC2 (Server)	192.168.20.20	255.255.255.0	192.168.20.1

5. ส่วนที่ 2 — การทดสอบการเชื่อมต่อ

5.1 ทดสอบ Ping จาก PC1 ไปยัง PC2

รันคำสั่งต่อไปนี้บน PC1:

```
ping 192.168.20.20
```

ผลลัพธ์ที่คาดหวัง: ได้รับ Reply จาก 192.168.20.20 ทุก Packet แสดงว่า Inter-VLAN Routing ทำงานได้ถูกต้อง

(ให้แนบ Screenshot ของผลลัพธ์ ping ในรายงานจริง)

5.2 การวิเคราะห์ ARP

ก่อนที่ Ping จะส่งได้สำเร็จ ระบบต้องทราบ MAC Address ของ Default Gateway (192.168.10.1) ก่อน กระบวนการ ARP จะเกิดขึ้นดังนี้:

1. PC1 ส่ง ARP Request: "Who has 192.168.10.1?" แบบ Broadcast ใน VLAN 10
2. Router R1 ตอบด้วย ARP Reply พร้อม MAC Address ของ Subinterface G0/0.10
3. PC1 เก็บ MAC ของ Gateway ใน ARP Cache
4. Packet จาก PC1 ถูกส่งไปยัง MAC ของ Gateway, Router ทำ Routing ไปยัง VLAN 20
5. Router ส่ง ARP Request หา PC2 ใน VLAN 20 (ถ้ายังไม่มีใน Cache)
6. PC2 ตอบ ARP Reply และ Router ส่ง Packet ไปยัง PC2

ตรวจสอบ ARP Cache บน PC1 ด้วยคำสั่ง arp -a

5.3 ตรวจสอบ Router

ตรวจสอบ Routing Table บน R1:

```
show ip route
```

ผลลัพธ์ที่คาดหวัง: มีเส้นทาง Directly Connected ไปยัง 192.168.10.0/24 และ 192.168.20.0/24

```
show ip interface brief
```

ผลลัพธ์ที่คาดหวัง: G0/0, G0/0.10, G0/0.20 มีสถานะ up/up

6. ส่วนที่ 3 — การติดตั้งและรัน MIME Server

6.1 เริ่มต้น Server บน PC2

เปิด Terminal บน PC2 แล้วรันคำสั่ง:

```
cd automation
python server/main_enhanced.py --host 0.0.0.0 --port 65432 --verbose
```

ข้อสังเกตสำคัญ: ต้อง Bind ที่ 0.0.0.0 ไม่ใช่ localhost (127.0.0.1) เพื่อให้รับ Connection จากเครือข่ายภายนอกได้

6.2 ส่งไฟล์จาก Client (PC1)

เปิด Terminal อีกหน้าต่างบน PC1 แล้วรัน:

```
cd automation
```

```
python client/main_enhanced.py assets/sample.png --host 192.168.20.20
--port 65432
```

กระบวนการที่เกิดขึ้น:

7. Client เปิด TCP Connection ไปยัง 192.168.20.20:65432
8. Client สร้าง JSON Header: {"mime_type": "image/png", "size": <ขนาดไฟล์>}
9. Client ส่ง JSON Header ตามด้วย Binary Data ของไฟล์
10. Server รับข้อมูล แยก JSON Header และบันทึกไฟล์

ยืนยันว่าไฟล์ถูกบันทึกใน Destination Folder บน Server ถ้าไม่สำเร็จให้ตรวจสอบ: Server bind ที่ 0.0.0.0 หรือไม่ / Firewall บล็อก Port 65432 หรือไม่ / IP Address และ Gateway ถูกต้องหรือไม่

7. ส่วนที่ 4 — การบันทึกและวิเคราะห์ Packet ด้วย Wireshark

7.1 การเริ่มต้นบันทึก

เปิด Wireshark บนทั้ง PC1 และ PC2 เลือก Network Interface ที่เชื่อมต่อกับ Switch ตั้ง Capture Filter:

```
tcp port 65432
```

จากนั้นรันการส่งไฟล์อีกครั้ง แล้วหยุดการบันทึกเมื่อเสร็จสิ้น บันทึกไฟล์เป็น client_capture.pcapng และ server_capture.pcapng

7.2 การวิเคราะห์ ARP

ค้นหา Packet ARP ในรายการ:

- ARP Request: "Who has 192.168.20.20?" — ส่งจาก PC1 ไปยัง Gateway
- ARP Reply: Router ตอบกลับด้วย MAC Address
- ตรวจสอบว่า MAC ที่ได้รับตรงกับ Router Interface G0/0.10

(แนบ Screenshot ARP Packet Details)

7.3 การวิเคราะห์ TCP Three-Way Handshake

ระบุ Packet สามรายการต่อไปนี้:

ลำดับ	Flag	ทิศทาง	ความหมาย
1	SYN	PC1 → PC2	Client ขอเชื่อมต่อ
2	SYN-ACK	PC2 → PC1	Server ยอมรับและยืนยัน
3	ACK	PC1 → PC2	Client ยืนยันการเชื่อมต่อ

บันทึก Sequence Number เริ่มต้นของทั้งสองฝั่ง (ISN: Initial Sequence Number) ซึ่งสุ่มขึ้นเพื่อความปลอดภัย (แนบ Screenshot)

7.4 การวิเคราะห์การส่งข้อมูล (Data Transfer)

คลิกขวาที่ Packet ใดก็ได้ในสตรีม → Follow → TCP Stream

คุณควรเห็น JSON Header ที่ส่งมาก่อน:

```
{"mime_type": "image/png", "size": 12345}
```

ตามด้วย Binary Data ของไฟล์ PNG (มักเริ่มด้วย Magic Bytes: 89 50 4E 47 = .PNG)

นี่คือหลักฐานที่แสดงว่า Application Layer Protocol ส่ง Structured Metadata ควบคู่กับ Binary Payload ผ่าน TCP Stream เดียวกัน

7.5 การวิเคราะห์ VLAN Tags (ถ้าบันทึกบน Trunk)

ถ้าบันทึก Traffic บนลิงก์ Trunk ให้มองหา Field:

- 802.1Q Virtual LAN — ใน Layer 2 Header
- VLAN ID: 10 (สำหรับ Traffic จาก PC1) หรือ 20 (สำหรับ Traffic จาก PC2)
- Priority: 0 (ค่า Default)

7.6 ตารางสรุปการวิเคราะห์ Packet

Layer	Field	ค่าที่พบ	ความหมาย
Ethernet	Src/Dst MAC	XX:XX:XX:XX	ทิศทางของ Frame
802.1Q	VLAN ID	10 หรือ 20	การแบ่งเครือข่ายออกเป็น Segment
IP	TTL	63 หรือ 127	ผ่าน Router 1 Hop (TTL ลดลง 1)
TCP	Flags	SYN/ACK/PSH	ความน่าเชื่อถือและการควบคุม
Payload	JSON Header	mime_type, size	Metadata ของไฟล์ที่ส่ง

8. ส่วนที่ 5 — การแก้ไขปัญหา (Troubleshooting Scenarios)

8.1 Scenario 1: Broken Routing — ปิด Subinterface

การดำเนินการ:

ปิด Subinterface G0/0.10 บน Router R1:

```
interface g0/0.10
shutdown
```

ผลลัพธ์ที่สังเกตพบ:

- Ping จาก PC1 ไปยัง PC2 ล้มเหลวทั้งหมด — แสดงข้อความ Request timeout
- Wireshark: แสดง SYN Retransmissions หลายครั้งโดยไม่ได้รับ SYN-ACK
- ไม่มี ARP Reply สำหรับ Default Gateway 192.168.10.1

การวิเคราะห์:

เมื่อ Subinterface ปิด Router ไม่สามารถรับ Packet จาก VLAN 10 ได้ ARP Request จาก PC1 ส่งออกไปแต่ไม่มีใครตอบ ทำให้ PC1 ไม่ทราบ MAC ของ Gateway และไม่สามารถส่ง IP Packet ออกนอก Subnet ได้

8.2 Scenario 2: Firewall Block — บล็อก Port

การดำเนินการ:

เพิ่ม Firewall Rule บน PC2 เพื่อบล็อก Port 65432 (Windows Firewall หรือ iptables บน Linux)

ผลลัพธ์ที่สังเกตพบ:

- TCP Client ส่ง SYN แต่ไม่ได้รับ SYN-ACK
- Wireshark: แสดง SYN Retransmissions ซ้ำๆ ด้วย Exponential Backoff
- Connection Timeout หลังจากพยายามหลายครั้ง

การวิเคราะห์:

Firewall Drop Packet ทำให้ Client ไม่ทราบว่า Port ถูกบล็อกหรือ Server ไม่มีอยู่ TCP จึงพยายาม Retransmit ซ้ำตามกลไก Exponential Backoff

8.3 Scenario 3: Wrong Gateway — กำหนด Gateway ผิด

การดำเนินการ:

เปลี่ยน Default Gateway ของ PC1 เป็น IP ที่ไม่ถูกต้อง เช่น 192.168.10.254

ผลลัพธ์ที่สังเกตพบ:

- Ping ล้มเหลว
- ARP Cache บน PC1: แสดง Gateway IP เป็น incomplete
- Wireshark: ARP Request ซ้ำๆ สำหรับ IP ที่ไม่มีอยู่จริง

การวิเคราะห์:

PC1 ส่ง ARP Request หา MAC ของ Gateway ที่กำหนดผิด แต่ไม่มีอุปกรณ์ใดตอบ ทำให้ ARP Incomplete และ Packet ทุกอย่างที่ต้องออกนอก Subnet ถูก Drop

8.4 Scenario 4: Packet Loss — จำลองการสูญหายของ Packet

การดำเนินการ:

ใช้ tc (Traffic Control บน Linux) เพื่อ Drop Packet 10%:

```
tc qdisc add dev eth0 root netem loss 10%
```

ผลลัพธ์ที่สังเกตพบ:

- Wireshark: แสดง TCP Retransmissions และ Duplicate ACK
- TCP Window Size ลดลง (Congestion Control)
- การส่งไฟล์ใช้เวลานานขึ้นแต่สำเร็จในที่สุด

การวิเคราะห์:

TCP ตรวจพบการสูญหายผ่าน Duplicate ACK (3 ACK ซ้ำ = Fast Retransmit) หรือ Timeout แล้วส่ง Segment ที่หายไปใหม่ Congestion Window ลดลงครึ่งหนึ่ง (Multiplicative Decrease) ทำให้ Throughput ลดลงชั่วคราว นี่คือ Reliability ของ TCP ที่ทำงานจริง

9. คำถามประเมินผลและคำตอบ

คำถามที่ 1: ทำไม TCP จึงส่ง Segment ที่สูญหายซ้ำ?

TCP เป็นโปรโตคอลแบบ Connection-oriented ที่รับประกันความน่าเชื่อถือในการส่งข้อมูล โดยกลไก Acknowledgment (ACK) — Receiver ส่ง ACK ยืนยันทุก Segment ที่ได้รับ ถ้า Sender ไม่ได้รับ ACK ภายใน Timeout Period หรือได้รับ Duplicate ACK 3 ครั้ง ก็จะส่ง Segment นั้นซ้ำ กลไกนี้ทำให้ TCP รับประกันการส่งข้อมูลครบถ้วนแม้เครือข่ายมีการสูญหายของ Packet

คำถามที่ 2: VLAN Tagging ส่งผลต่อ Broadcast Domain อย่างไร?

VLAN แบ่งเครือข่ายออกเป็น Broadcast Domain ที่แยกจากกัน Packet ที่มี Tag VLAN 10 จะถูก Flood เฉพาะใน Port ที่เป็นสมาชิก VLAN 10 เท่านั้น ไม่ข้ามไป VLAN 20 ทำให้ลด Broadcast Traffic โดยรวม เพิ่มความปลอดภัย และแยก Segment ของเครือข่ายทางตรรกะ

คำถามที่ 3: MIME ต่างจาก TCP Header อย่างไร?

TCP Header อยู่ใน Transport Layer ทำหน้าที่ควบคุมการส่งข้อมูล เช่น Sequence Number, Port Number, Flags ส่วน MIME เป็น Application Layer Metadata ที่อธิบายความหมายของข้อมูล เช่น ประเภทไฟล์ ขนาด TCP ไม่รู้ว่า Payload เป็นรูปภาพหรือเอกสาร — MIME เป็นผู้บอก Meaning ให้กับ Application

คำถามที่ 4: การตัดสินใจ Routing เกิดขึ้นที่ไหน?

การตัดสินใจ Routing เกิดขึ้นที่ Router R1 ใน Layer 3 (Network Layer) เมื่อ Packet มาถึง Router จะดู Destination IP Address เทียบกับ Routing Table แล้วส่ง Packet ออก Interface ที่เหมาะสม (G0/0.10 หรือ G0/0.20) ในกรณีนี้ Router ทำงานผ่าน Subinterface แบบ Router-on-a-Stick

คำถามที่ 5: ทำไม ARP จึงต้องเกิดขึ้นก่อน TCP?

Layer 2 (Ethernet) ต้องการ MAC Address เพื่อส่ง Frame ไปยังอุปกรณ์ถัดไปในเครือข่าย แต่ Application รู้เพียง IP Address ของ Destination ARP จึงทำหน้าที่แปลง IP Address → MAC Address ก่อน TCP จะเริ่ม Three-Way Handshake ได้ ระบบต้องทราบ MAC ของ Next Hop (Default Gateway) เสียก่อน

10. Reflection — ความหมายของ Application ผ่านชั้น Encapsulation

การทดลองนี้แสดงให้เห็นเส้นทางที่ไฟล์ภาพ (PNG) เดินทางจาก PC1 ถึง PC2 ผ่านกระบวนการต่อไปนี้:

ขั้นตอน	Layer / กระบวนการ	สิ่งที่เกิดขึ้น

1	Application	Client สร้าง JSON Header + อ่าน Binary File — "intention"
2	Transport (TCP)	แบ่งข้อมูลเป็น Segment เพิ่ม Sequence Number — "bytes"
3	Network (IP)	เพิ่ม IP Header, Router ตัดสินใจเส้นทาง — "routing"
4	Data Link (802.1Q)	เพิ่ม Ethernet Frame + VLAN Tag — "frames"
5	Physical	แปลงเป็นสัญญาณไฟฟ้า/แสง — "signals"
6	De-encapsulation	Server ถอด Layer ทีละชั้น กู้คืนข้อมูลต้นฉบับ — "meaning"

สิ่งที่น่าสังเกตที่สุดคือ "ความหมาย" (Meaning) ไม่ได้หายไปแม้ข้อมูลจะถูกห่อหุ้มด้วย Header จากหลายชั้น TCP รับประกันว่าทุก Byte มาถึงครบ IP รับประกันว่าข้อมูลเดินทางถูกทิศทาง และ MIME Header รับประกันว่า Server รู้ว่ากำลังรับอะไร กระบวนการนี้คือแก่นแท้ของระบบเครือข่ายคอมพิวเตอร์

11. ข้อผิดพลาดที่พบบ่อยและวิธีแก้ไข

ปัญหาที่พบ	สาเหตุ	วิธีแก้ไข
Server ไม่รับ Connection	Bind ที่ localhost (127.0.0.1) แทน 0.0.0.0	ใช้ --host 0.0.0.0
Ping ข้าม VLAN ไม่ได้	ไม่มี Trunk Link หรือ VLAN ไม่ถูก Allow	ตรวจสอบ show interfaces trunk
Subinterface ไม่ทำงาน	Physical Interface ยังไม่ no shutdown	รัน no shutdown บน G0/0
Connection ถูกบล็อก	OS Firewall เปิดอยู่	ปิด Firewall หรือเพิ่ม Rule
Wireshark ไม่แสดง Packet	เลือก Interface ผิด	เลือก Interface ที่เชื่อมต่อ Switch
VLAN Tag ไม่ปรากฏใน Wireshark	บันทึกที่ Access Port แทน Trunk	บันทึกบน Trunk Interface

12. สรุปและข้อเสนอแนะ

12.1 สรุปผลการทดลอง

การทดลองนี้ประสบความสำเร็จในการสาธิตการทำงานร่วมกันของหลาย Layer ในระบบเครือข่าย ตั้งแต่การ Routing ระหว่าง VLAN ไปจนถึงการส่งข้อมูลระดับ Application ผู้เรียนได้เห็นว่า ARP, TCP, IP, VLAN และ MIME Protocol แต่ละชิ้นส่วนทำงานอย่างไรและเชื่อมโยงกันอย่างไรในสถานการณ์จริง

12.2 ข้อเสนอแนะสำหรับการทดลองต่อไป (Extension Activities)

- เพิ่ม TLS Encryption เพื่อปกป้องข้อมูลระหว่างการส่ง
- วัด Throughput เทียบกับ Latency ภายใต้สภาวะต่างๆ
- บันทึกและวิเคราะห์ Checksum Errors

- เพิ่ม Checksum Field ใน Custom Protocol
- Containerize Server ด้วย Docker
- เปรียบเทียบกับการส่งไฟล์ผ่าน HTTP

12.3 บทเรียนสำคัญ

การทดลองนี้ไม่ได้สอนเพียงแค่เครือข่าย แต่สอนหลักการพื้นฐานสี่ประการ:

11. Protocol Layering Philosophy — แต่ละชั้นมีหน้าที่เฉพาะและโปร่งใสต่อชั้นอื่น
12. Reliability Engineering — TCP รับประกันการส่งข้อมูลแม้เครือข่ายไม่สมบูรณ์
13. Metadata Semantics — ข้อมูลต้องการคำอธิบายเพื่อให้มีความหมาย
14. Distributed System Fundamentals — ระบบจริงสร้างบนรากฐานเหล่านี้

ดังที่ Lab นี้แสดงให้เห็น: ไฟล์หนึ่งไฟล์เดินทางในรูปแบบ intention → bytes → frames → signals → bytes → meaning กระบวนการนั้นคือแก่นแท้ของการสื่อสารดิจิทัล